

Reviewer Pack — Kronecker Coefficient Gap in Set Disjointness Communication ...

Entry d82cb6cd01e1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 23:12:08 UTC

Kronecker Coefficient Gap in Set Disjointness Communication Complexity

Entry ID: d82cb6cd01e1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 23:12:08 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups (Kronecker coefficients) **Field B** (complexity object): Communication Complexity of Set Disjointness

Statement:

For any set disjointness instance with n elements, the Kronecker coefficient $\kappa(\lambda, \mu, \nu)$ for partitions λ, μ, ν of size n satisfies $\kappa(\lambda, \mu, \nu) \geq (\log n)^{\Omega(1)}$ when the communication matrix has rank $\Omega(n^2)$.

Rationale (proposer's reasoning):

Kronecker coefficients encode decomposition multiplicities of tensor products, which might mirror the combinatorial structure of disjointness matrices. Their growth could correlate with matrix rank, revealing hidden symmetry constraints in communication protocols.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a642f1b070dcce68

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_disjoint_sets(n):
        sets = []
        for _ in range(2):
            elements = set(range(n))
            while len(elements) > 0:
                size = random.randint(1, min(len(elements), n // 4))
                subset = random.sample(elements, size)
                sets.append(subset)
                elements -= set(subset)
        return sets

    def matrix_rank(matrix):
        m, n = len(matrix), len(matrix[0])
        rank = 0
        for i in range(m):
            if any(matrix[i][j] != 0 for j in range(n)):
                rank += 1
                for j in range(n):
                    if matrix[i][j] != 0:
                        factor = matrix[i][j]
                        for k in range(i, m):
                            matrix[k][j] /= factor
                        break
                for k in range(m):
                    if k != i and any(matrix[k][j] != 0 for j in range(n)):
                        for j in range(n):
                            matrix[k][j] -= matrix[i][j]
        return rank

    def young_diagram_partitions(n):
        partitions = []
        def generate_partition(k, n, partition):
            if k == 0:
```

```

        partitions.append(partition[:])
        return
    for i in range(min(n, k), -1, -1):
        partition.append(i)
        generate_partition(k - i, n - i, partition)
        partition.pop()
    generate_partition(n, n, [])
    return partitions

def kronecker_coefficient( $\lambda$ ,  $\mu$ ,  $v$ ):
    if len( $\lambda$ ) != len( $\mu$ ) or len( $\mu$ ) != len( $v$ ):
        return 0
    m = len( $\lambda$ )
     $\lambda$  = [ $\lambda[i]$  - i for i in range(m)]
     $\mu$  = [ $\mu[i]$  - i for i in range(m)]
     $v$  = [ $v[i]$  - i for i in range(m)]
    n = sum( $\lambda$ )
    if any(x < 0 or x > n for x in  $\lambda$  +  $\mu$  +  $v$ ):
        return 0
    def hook_length(p, q):
        return (p + 1) * (q + 1) // 2 - p - q
    def sign(p):
        s = 1
        for i in range(1, len(p)):
            for j in range(i):
                if p[i] < p[j]:
                    s *= -1
        return s
    def hook_length_product(partition):
        product = 1
        for i in range(len(partition)):
            for j in range(partition[i]):
                product *= hook_length(i, j)
        return product
    def young_symmetrizer(partition):
        n = sum(partition)
        matrix = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if partition[j] > partition[i]:
                    matrix[i][j] = 1
                elif partition[j] < partition[i]:
                    matrix[i][j] = -1
        return matrix
    def kronecker_coefficient_recursive( $\lambda$ ,  $\mu$ ,  $v$ ):
        if len( $\lambda$ ) == 0:
            return 1
        m = len( $\lambda$ )
         $\lambda_m$  =  $\lambda[-1]$ 
         $\mu_m$  =  $\mu[-1]$ 
         $v_m$  =  $v[-1]$ 
         $\lambda$  =  $\lambda[:-1]$ 
         $\mu$  =  $\mu[:-1]$ 
         $v$  =  $v[:-1]$ 
        result = 0
        for i in range(max(0,  $\mu_m$  -  $v_m$ ), min( $\lambda_m$ ,  $\mu_m$ ) + 1):

```

```

        result += sign(i) * hook_length_product([λm - i] + λ) * hook_length_product([μm - i] + μ)
    return result
return abs(kronecker_coefficient_recursive(λ, μ, v))

n = random.randint(5, 40)
sets = generate_disjoint_sets(n)
matrix = [[1 if i in set_ else 0 for j in range(n)] for set_ in sets]
rank = matrix_rank(matrix)
partitions = young_diagram_partitions(n)
min_kronecker_coefficient = float('inf')
for λ in partitions:
    for μ in partitions:
        for v in partitions:
            coeff = kronecker_coefficient(λ, μ, v)
            if coeff < min_kronecker_coefficient:
                min_kronecker_coefficient = coeff
conjecture_holds = min_kronecker_coefficient >= (math.log(n)) ** 1.5
counterexample = "" if conjecture_holds else f"Minimum Kronecker coefficient {min_kronecker_coefficient}"

return {
    "metric_name": "Kronecker Coefficient Gap",
    "metric_value": min_kronecker_coefficient,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dla610cc.py", line 143, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dla610cc.py", line 115, in run_trial
    sets = generate_disjoint_sets(n)
           ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dla610cc.py", line 27, in generate_disj
    subset = random.sample(elements, size)
              ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence. ")
TypeError: Population must be a sequence.  For dicts or sets, use sorted(d).
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to population sequence error, preventing data collection. Support condition cannot be evaluated without valid results. | next: Fix population sequence handling in generate_disjoint_sets() by converting inputs to lists before sampling

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	20814
2	preregistration	ollama_remote	qwen3:8b	0	0	25868
3	novelty	ollama_remote	qwen3:8b	0	0	22025
4	novelty	ollama_remote	qwen3:8b	0	0	13671
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18947
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16416
7	judge	ollama_remote	qwen3:8b	0	0	17494

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 135235 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/d82cb6cd01e1.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/d82cb6cd01e1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d82cb6cd01e1.tar.gz` (if generated)